



Apache Spark Week-5 Q&A

Big Data Processing, PySpark DataFrames & Data Cleaning

Q1. What are the key limitations of traditional MapReduce that make Spark a preferred choice for modern big data processing?

Answer:

Traditional MapReduce has several limitations:

- It stores intermediate results on disk after every stage, making processing slow.
- It is not suitable for iterative algorithms used in Machine Learning.
- Writing MapReduce programs is complex because separate Map and Reduce functions are required.
- It has high disk I/O overhead.
- Real-time data processing is difficult.

Advantages of Spark:

- In-memory processing (stores data in RAM)
- Much faster (up to 100× faster in memory)
- Easy-to-use APIs
- Supports SQL, Machine Learning, Graph Processing, and Streaming
- Better performance for iterative computations

Q2. Explain how Spark uses In-Memory Computing to speed up iterative machine learning algorithms compared to disk-based systems.

Answer:

Spark stores intermediate data in memory (RAM) instead of writing it to disk after every operation.

In Machine Learning algorithms, the same dataset is processed multiple times. Since Spark keeps the data in memory, repeated disk reads and writes are avoided, which significantly reduces execution time.

Benefits:

- Faster execution
- Lower disk I/O
- Better performance for iterative algorithms
- Efficient resource utilization

Q3. Write a code snippet to remove all duplicate rows from a DataFrame based on user_id and transaction_date.**Answer:**

```
df = df.dropDuplicates(["user_id", "transaction_date"])
```

This removes duplicate rows based only on the specified columns.

Q4. Given a DataFrame df_sales, write a query to filter rows where the region is 'West' and group by product_category to find the average sale_amount.**Answer:**

```
from pyspark.sql.functions import avg

df_sales.filter(df_sales.region == "West") \
    .groupBy("product_category") \
    .agg(avg("sale_amount").alias("Average_Sales")) \
    .show()
```

Q5. What is the difference between .na.drop() and .na.fill()? Provide an example.**Answer:**

.na.drop()	.na.fill()
Removes rows containing null values.	Replaces null values with a specified value.

Example:

```
df = df.na.fill({"status": "Unknown"})
```

This replaces all null values in the status column with "Unknown".

Q6. Write a query to find the total count of records for each city where the count is greater than 100.

Answer:

```
from pyspark.sql.functions import count

df.groupBy("city") \
    .count() \
    .filter("count > 100") \
    .show()
```

Q7. How does the immutability of Spark DataFrames affect data cleaning?

Answer:

Spark DataFrames are immutable, meaning they cannot be modified after creation.

Whenever a cleaning operation such as dropping or renaming columns is performed, Spark creates a new DataFrame instead of changing the original one.

Example:

```
new_df = df.drop("salary")
```

The original DataFrame remains unchanged.

Benefits:

- Data safety
- Fault tolerance
- Easier debugging
- Parallel processing support

Q8. Write a Spark command to filter rows where age is between 18 and 30 and subscription is 'Premium'.

Answer:

```
df.filter(
    (df.age >= 18) &
    (df.age <= 30) &
    (df.subscription == "Premium")
).show()
```

Q9. Why should null values be handled before mathematical aggregations?

Answer:

Null values can affect aggregation results.

For example:

- Average may become incorrect.
- Sum may ignore missing values.
- Calculations can produce unexpected results.

Handling null values before aggregation ensures:

- Accurate calculations
- Reliable analytics
- Better data quality

Q10. Write code to cast raw_timestamp to TimestampType and rename it to event_time.**Answer:**

```
from pyspark.sql.types import TimestampType
from pyspark.sql.functions import col

df = df.withColumn(
    "event_time",
    col("raw_timestamp").cast(TimestampType())
).drop("raw_timestamp")
```

Q11. Explain the Shuffle process. Why is it a wide transformation?**Answer:**

Shuffle is the process of redistributing data across partitions during operations such as:

- groupBy()
- join()
- distinct()
- reduceByKey()

It is called a wide transformation because data from multiple partitions must be exchanged over the network.

Disadvantages:

- Network communication
- Increased execution time
- Higher memory usage

Q12. Write code to remove rows where email is null OR username is empty.**Answer:**

```
from pyspark.sql.functions import col

df = df.filter(
```

```
col("email").isNotNull()  
) .filter(  
    col("username") != ""  
)
```

Q13. How do you use .agg() to calculate min, max and mean of the price column?

Answer:

```
from pyspark.sql.functions import min, max, avg  
  
df.agg(  
    min("price").alias("Minimum"),  
    max("price").alias("Maximum"),  
    avg("price").alias("Average")  
) .show()
```

Q14. What is the risk of using inferSchema=true with inconsistent date formats?

Answer:

When the dataset contains inconsistent date formats, inferSchema=true may infer incorrect data types.

Risks:

- Dates may be treated as strings.
- Parsing errors may occur.
- Incorrect sorting and filtering.
- Aggregations based on dates may fail.

It is safer to define the schema explicitly when data is inconsistent.

Q15. Write a final processing pipeline that:

1. Filters duplicates
2. Fills null prices with 0
3. Groups by store_id to calculate total revenue

Answer:

```
from pyspark.sql.functions import sum  
  
result = (  
    df.dropDuplicates()  
    .na.fill({"price":0})  
    .groupBy("store_id")  
    .agg(sum("price").alias("Total_Revenue"))  
)
```

```
result.show()
```

Explanation:

- Step 1: `dropDuplicates()` removes duplicate records.
- Step 2: `na.fill({"price":0})` replaces null prices with 0.
- Step 3: `groupBy("store_id")` groups data by store.
- Step 4: `sum("price")` calculates the total revenue for each store.